THE C GAME HUB

A Multi-Logic Gaming Suite in C

Subject: Foundation of Programming

Submitted By:

Tanisk Sinha – 1262250519

Vinay Borse – 1262250653

Pratyush Dey – 1262252536

Raghav Shelke – 1262250833

Jeet Zanvar – 1262250354

Institution: MIT-WPU, Pune

TABLE OF CONTENTS

Acknowledgement 3

1. INTRODUCTION 4

2. PROBLEM STATEMENT 4

3. OBJECTIVES 5

4. PROJECT DESCRIPTION 5

4.1 Number Guessing Game 5

4.2 Rock Paper Scissors 5

4.3 Guess the Word 5

5. SYSTEM DESIGN AND METHODOLOGY 6

5.1 Procedural Approach 6

5.2 Randomization Algorithm 6

5.3 Input Buffer Management 7

6. CODE 7

7. RESULTS AND DISCUSSION 11

8. CONCLUSION 11

9. FUTURE SCOPE 11

Acknowledgement

We would like to express our deepest gratitude to our project guide and faculty at MIT-WPU for their unwavering support and technical insights. This project was a cumulative effort that required applying theoretical knowledge from the 'Foundation of Programming' course into a practical, user-facing application. We thank the Computer Science department for providing the environment necessary to explore low-level systems logic and procedural programming.

Special thanks go to our team members for their collaborative spirit and dedication. Balancing different programming modules—from string manipulation to mathematical randomization—required constant communication and rigorous peer review of the source code. The synergy between team members allowed us to tackle complex logical hurdles and ensure the robustness of the final software hub.

1. INTRODUCTION

The 'C Game Hub' is a console-based multi-game application designed to demonstrate the power of the C Standard Library. In an era of complex graphical engines, this project returns to the fundamentals to highlight how logic, memory management, and clean control flows can create engaging experiences. The hub serves as a centralized station for games involving mathematical guessing, probability-based decision making, and string processing. By integrating multiple modules into a single interface, the project showcases the versatility of procedural programming in C.

2. PROBLEM STATEMENT

Most entry-level C programs suffer from two main issues: lack of modularity and poor input validation. When a user enters a character in a field expecting an integer, standard C programs often enter infinite loops or crash. Furthermore, generating truly varied outcomes requires careful seeding of random number generators. This project addresses these issues by creating a robust, modular system capable of handling diverse data types and user-input 'traps'. The core challenge was to ensure that each game could run independently while sharing a common input/output infrastructure.

3. OBJECTIVES

The primary objectives of this project include:

- Implementing a centralized menu using while(1) loops and switch-case logic.
- Developing a 'Number Guessing' module utilizing time-seeded randomization.
- Creating a 'Rock Paper Scissors' engine with multi-path win/loss conditions.
- Designing a 'Guess the Word' game to practice string handling and case-sensitivity logic.
- Ensuring program stability via custom input-buffer clearing functions.

4. PROJECT DESCRIPTION

4.1 Number Guessing Game

This game challenges the user to identify a hidden number between 1 and 100. It utilizes the rand() function, seeded with time(NULL) to ensure a different number every session. The game provides real-time feedback—'Higher' or 'Lower'—helping the user narrow down the correct answer while tracking the total number of attempts used. The implementation includes checks to ensure the user stays within the 1-100 range and provides feedback for out-of-bounds guesses.

4.2 Rock Paper Scissors

A classic game of strategy and chance. The core logic involves a 3-way conditional comparison. Numerical inputs (1, 2, 3) are mapped to Rock, Paper, and Scissors. The computer's choice is generated via modulo arithmetic on a random integer. The win conditions are evaluated using logical AND/OR operators to determine a Draw, User Win, or Computer Win. This game highlights the effective use of switch-case structures to manage discrete game states and user choices.

4.3 Guess the Word

This module transitions from numbers to strings. It manages a dictionary of words (primarily fruits) and corresponding hints. It demonstrates the use of char arrays, the tolower() function for case-insensitive comparison, and the strcmp() function for validation. This game tests the user's vocabulary and the developer's ability to manage string memory safely. The addition of hints provides an extra layer of engagement, making the game more accessible to players.

5. SYSTEM DESIGN AND METHODOLOGY

5.1 Procedural Approach

The system is designed using Top-Down modular programming. Each game exists as a standalone void function, which is called by the main controller. This makes the code highly readable and allows for future games to be added without disturbing existing logic. The procedural nature of C is leveraged to create a clear separation between game logic, input handling, and user feedback.

5.2 Randomization Algorithm

To achieve non-deterministic behavior, the program uses srand(time(NULL)). This takes the current epoch time as a seed. Without this, the rand() function would generate the same sequence of numbers every time the program restarts, making the games predictable. This is a fundamental technique in computer science for simulating chance and variability in software applications.

Example Logic Flow for RPS:

User Input -> Computer Choice (rand % 3) -> Comparison Matrix -> Output Result.

5.3 Input Buffer Management

A critical feature of the 'C Game Hub' is the clearInputBuffer() function. It uses a while loop with getchar() to flush the stdin stream. This ensures that if a user accidentally types multiple characters or symbols when a number is expected, the extra characters do not 'leak' into the next prompt, which is a common cause of CLI bugs. This ensures a smooth and error-free user experience even when invalid data is provided.

6. CODE

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <ctype.h>

void clearInputBuffer() {
    int c;
    while ((c = getchar()) != '\n' && c != EOF);
}

void playNumberGuess() {
    int secret = (rand() % 100) + 1, guess, tries = 0;
    printf("\n[Number Guess] I'm thinking of 1-100.\n");

    do {
        printf("Your guess: ");
        if (scanf("%d", &guess) != 1) {
            printf("Please enter a number!\n");
            clearInputBuffer();
            continue;
        }
        tries++;
        if (guess > secret) printf("lower!\n");
        else if (guess < secret) printf("higher!\n");
    } while (guess != secret);

    printf("Bingo! You got it in %d tries.\n", tries);
}

```

```c
void playRPS() {
    int user, computer = (rand() % 3) + 1;
    printf("\n[RPS] 1:Rock, 2:Paper, 3:Scissors: ");

    if (scanf("%d", &user) != 1) {
        printf("Invalid input.\n");
        clearInputBuffer();
        return;
    }

    if (user < 1 || user > 3) {
        printf("Invalid choice! Pick 1, 2, or 3.\n");
        return;
    }

    char *choices[] = {"", "Rock", "Paper", "Scissors"};
    printf("You: %s | Computer: %s\n", choices[user], choices[computer]);

    if (user == computer) printf("Draw!\n");
    else if ((user == 1 && computer == 3) ||
             (user == 2 && computer == 1) ||
             (user == 3 && computer == 2))
        printf("You Win!\n");
    else
        printf("Computer Wins!\n");
}
```

```c
void playGuessWord() {
    char *words[] = {
        "apple", "banana", "orange", "grape", "mango",
        "cherry", "peach", "lemon", "berry", "melon"
    };

    char *hints[] = {
        "A common red fruit",
        "A long yellow fruit",
        "A citrus fruit that's also a color",
        "Small fruits that grow in bunches",
        "A tropical fruit with orange flesh",
        "Small red fruits used on desserts",
        "A fuzzy fruit with a pit",
        "A sour yellow citrus fruit",
        "A small sweet fruit (generic)",
        "Large fruit with green rind"
    };

    int count = sizeof(words) / sizeof(words[0]);
    int index = rand() % count;

    char guess[20];
    printf("\n[Guess the Word]\nHint: %s\n", hints[index]);
    printf("Enter word: ");
    scanf("%19s", guess);

    for (int i = 0; guess[i]; i++)
        guess[i] = tolower(guess[i]);

    if (strcmp(guess, words[index]) == 0)
        printf("Correct! 🎉 It was %s.\n", words[index]);
    else
        printf("Wrong! The answer was %s.\n", words[index]);

    clearInputBuffer();
}
```

```c
int main() {
    int choice;
    srand(time(NULL));

    while (1) {
        printf("\n====================");
        printf("\n   THE C GAME HUB");
        printf("\n====================");
        printf("\n1. Number Guessing");
        printf("\n2. Rock Paper Scissors");
        printf("\n3. Guess the Word");
        printf("\n4. Exit");
        printf("\nPick a game: ");

        if (scanf("%d", &choice) != 1) {
            printf("Invalid input. Enter a number.\n");
            clearInputBuffer();
            continue;
        }

        if (choice == 4) break;

        switch (choice) {
            case 1: playNumberGuess(); break;
            case 2: playRPS(); break;
            case 3: playGuessWord(); break;
            default: printf("Try again (1-4).\n");
        }
    }

    printf("Goodbye!\n");
    return 0;
}
```

7. RESULTS AND DISCUSSION

The project successfully met all technical requirements. The menu system is stable, and games provide immediate, accurate feedback. Testing showed that the input validation can handle non-numeric entries without crashing, fulfilling the goal of a robust user experience. User feedback indicated that the system is intuitive and provides an enjoyable way to interact with basic programming concepts.

8. CONCLUSION

The 'C Game Hub' demonstrates that foundational programming concepts like loops, switch-cases, and string manipulation are sufficient to build complex, interactive software. The project reinforced our understanding of procedural logic and error-handling in C. By successfully integrating different types of game logic, we have gained valuable experience in modular software design and user-centric application development.

9. FUTURE SCOPE

Future versions will focus on implementing a Graphical User Interface (GUI) and adding file-handling capabilities to save user profiles and persistent high-score leaderboards. We also plan to introduce more complex games like Tic-Tac-Toe and Quiz modules to further expand the hub.